

BioFast: An Efficient Privacy-Preserving Face Verification Protocol From FHE and Cryptographic Hash Functions

Parhat Abla¹, Rongbin Huang, Yongqiang Li², and Mingsheng Wang

Abstract—With the popularity of the mobile Internet and smart-phones, remote facial recognition is increasingly becoming a new way of verifying identity. Due to privacy concerns in the face recognition protocols, cryptographic tools (e.g., FHE schemes, garbled circuits) are used to protect the facial information. However, the existing protocols use homomorphic encryption schemes in a black-box way, which strongly affects the protocol’s performance. Furthermore, the garbled circuit-based face recognition protocols need multiple interactions between the participants, which results in more inefficient protocols. Motivated by the above problems, we propose a more efficient privacy-preserving face verification protocol by tailoring multiple newly designed techniques, which may be of independent interest and may find further applications in related biometric verification protocols. In particular, we introduce a new packing technique that can homomorphically compute multiple inner products simultaneously. Compared with the previous Chinese Remainder Theorem (CRT)-based methods, our packing method works on all the modulus of the ring $\mathbb{Z}_q[x]/(x^n + 1)$. Furthermore, we design a cryptographic hash-based privacy-preserving non-interactive threshold comparison scheme with significantly better communication latency than the garbled circuit-based approach. Applying the above two independent, valuable techniques, our privacy-preserving face verification protocol achieves better space and time efficiency than the previous protocols. Our theoretical analyses are supported by the experimental evaluations.

Index Terms—Privacy-preserving, face recognition, FHE, hash.

I. INTRODUCTION

FACE verification is the process of verifying if an inputted facial probe feature (usually from a client) matches a stored facial feature template on the server. Thus, this kind of recognition has rich applications in identification scenarios. However,

Received 23 October 2024; revised 8 July 2026; accepted 9 July 2026. Date of publication 21 July 2026; date of current version 1 September 2026. The work of Parhat Abla was supported by the STU Scientific Research Initiation under Grant NTF25015T. The work of Yongqiang Li was supported by the National Natural Science Foundation under Grant 12371525. This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant XDB0690200. (Corresponding authors: Yongqiang Li; Mingsheng Wang.)

Parhat Abla is with the College of Mathematics and Computer Science, Shantou University, Shantou 100190, China (e-mail: parhat@stu.edu.cn).

Rongbin Huang is with the School of Software, South China Normal University, Guangzhou 528225, China (e-mail: huangrongbin@m.scnu.edu.cn).

Yongqiang Li and Mingsheng Wang are with the State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, 100045 Beijing, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100045, China (e-mail: liyongqiang@iie.ac.cn; wangmingsheng@iie.ac.cn).

Digital Object Identifier 10.1109/TDSC.2026.3715522

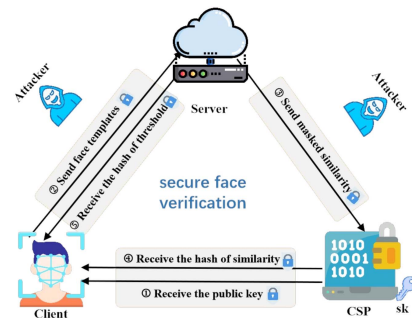


Fig. 1. A three party facial verification protocol.

there is a security concern that the server may be hacked, or the server may not even be honest. In this case, the stored facial features may leak to the adversarial party, who may use this facial information to emulate the facial feature’s owner. Therefore, the privacy-preserving facial recognition protocols [1], [2], [3], [4] are widely studied.

An intuitive way to solve the above problem is to use homomorphic encryptions (HE). In particular, the template facial features are encrypted by the client at the registration phase and stored on the server. When a client makes verification by sending the ciphertext of the probe facial feature, the server homomorphically computes the similarity between the probe feature and the stored feature template. Then, the result is sent back to the client. Then, the client obtains the similarity by decrypting the received similarity and then comparing it with the public threshold to get the final verification result. This two-party (client-server) approach works well if the client has the decryption secret key of the HE scheme. However, in many face verification scenarios, the client only has the public key and needs to prove its identity through facial features. Thus, the above simple approach only partially solves the previous security problem.

To completely solve the previous security concern, the cryptographic service provider (CSP) is introduced [5] in verification protocol just as shown in Fig. 1, where the CSP has the decryption key of the HE scheme. Now, the homomorphically computed masked similarity¹ and then sent to the CSP, and then

¹If the plaintext similarity is sent to CSP, then the CSP can obtain the similarity and may further get more information about the probe feature. Therefore, the masked similarity is sent to the CSP

the three parties interact to ensure only the client gets nothing but the verification result. Although this three-party approach can securely authenticate the face identities, the existing protocols have drawbacks in the similarity-computation and threshold-similarity comparison phases, as we elaborate next.

In the similarity-computation phase, previous works [5], [6], [7], [8] use the FHE schemes in a black-box way, and thus, the similarity computation process mainly relies on the efficiency of the FHE scheme. More precisely, the existing works use batching techniques of underlying homomorphic encryptions to encode features (vectors). However, inherited from the Chinese Remainder Theorem (CRT) used in batch techniques², the modulus of the FHE schemes [9], [10] should be large enough to embed the whole feature (vector) into one ciphertext. The larger FHE scheme modulus decreases its security. Thus, to achieve the expected security level with the above larger modulus, the other parameters should be appropriately large. This comes up with a more extensive communication overhead in the verification protocol. On the other hand, to use the CRT packing technique of the lattice-based FHE schemes, the scheme modulus q should satisfy $q^d \bmod 2n \equiv 1$ to pack d elements. Therefore, the CRT packing does not hold for all the ring modulus.

In the threshold-similarity comparison phase, the previous work of [11], [12] used homomorphic encryption-based techniques in an interactive way. Therefore, the underlying comparison process heavily relies on the inefficiency of the underlying homomorphic encryption scheme. Besides, the works [5], [8] used the garbled circuit-based approach. However, the garbled gate evaluation process needs to use oblivious transfer (OT), and the garbled circuit-based protocol needs more frequent interactions between the CSP and the server to evaluate the garbled threshold-similarity comparison circuit. Thus, the communication network's latency strictly affects the verification protocol's efficiency. As we show in the experimental evaluations section of this paper, sending garbled tables and their labels in a local area network (LAN) environment takes about 10X more time than in the local evaluation. In addition, to achieve desired performances, the existing garbled circuit protocols and DGK protocol-based approaches are based on the classic number-theoretic assumptions and thus can be attacked by quantum computers.

Motivated by the deficiencies in the existing protocols mentioned above, our goal in this paper is to propose an efficient privacy-preserving face verification protocol in the three-party model. To efficiently compute the similarity, we introduce a new coefficient embedding method instead of using the CRT-based technique as in the previous protocols. To achieve latency-robust threshold-similarity comparison, we construct a cryptographic hash function-based sub-protocol. Our contributions are presented below.

A. Our Contributions

In this paper, we propose a more efficient privacy-preserving face verification protocol by tailoring multiple newly designed

methods, which may be of independent interest and can be applied to other biometric verification protocols straightforwardly, to improve the efficiency of the state-of-the-art biometric verification (like fingerprint, iris) protocols. Our main contributions are as follows:

- We design a novel coefficient-based packing technique for the inner product of multiple feature vectors. Our packing method can pack multiple features into one FHE ciphertext and thus can compute multiple inner products simultaneously. Our batch computing method can be applied for any ring modulus q of the cyclotomic ring $\mathbb{Z}_q[x]/x^n + 1$. This cyclotomic ring is very important since the lattice-based homomorphic encryption schemes are designed on this kind of ring. The previous Chinese Remainder Theorem (CRT) method (FHE packing) only applies over the special modulus of this ring. Therefore, our technique brings a more general tool for computing inner products simultaneously. Moreover, the proposed packing method achieves substantial improvements in computational performance. In our experiments, it takes 1.524 ms for a similarity computation process based on the proposed packing technique, which is about $5\times$ faster than the CRT-based approaches [5], [6], [13], [14].
- We design a novel hash-based privacy-preserving threshold-similarity comparison scheme. Our hash-based threshold comparison is concise but useful. More precisely, there is no interaction between the CSP and the server in our hash-based threshold comparison process. Our hash-based threshold-similarity comparison process only needs about 0.1 millisecond, which is almost $10\times$ better than that of garbled circuit-based approaches [5], [8]. In addition, the total communication overhead is about 2.5 KB, which is about $3\times$ better than that of garbled circuit-based approaches [5], [8].
- We propose a more efficient privacy-preserving face recognition protocol from FHE and cryptographic hash functions. Our protocol can secure the verification result from the server and the CSP. In addition, the cosine similarity is also protected from the semi-honest client, CSP, and the server. With the help of the above two independent, useful techniques, our protocol achieves better space and time efficiency than the previous garbled-based protocols. In particular, during the enrollment phase, the proposed protocol takes 1.6346 ms and 217.35 KB communication overhead for each face enrollment. During the verification phase, the proposed protocol takes 2.7433 ms and 111.589 KB communication overhead for each verification. In addition, our protocol achieves post-quantum security.

We compare our work with the related techniques in Table I. Furthermore, to support our above ideas, we use the SEAL library and SHA256 to evaluate the performance of the proposed protocol and each phase. Experimental results show that our protocol is more efficient than the state-of-the-art methods.

II. RELATED WORK

The biometric recognition schemes are widely studied in the works of literature. Schroff et al. [18] proposed a face

²Since the NTT-based packing is the special case of the CRT-based packing, it belongs to the category of the CRT-based packings. Thus, we also use the notion CRT-based technique to denote the NTT-based packing.

TABLE I
COMPARISON AMONG THE PRIVACY-PRESERVING FACE VERIFICATIONS USING CRYPTOGRAPHIC TECHNIQUES

Solutions	Technique	Similarity	NoLeakS	NoLeakC	Batching [‡]	NoRotate	Secure NI Compare	Post-Quantum [#]
Two-party Models								
Pastoriza et al. [15]	HE	ED	✓	✗	✗	N/A	✗	✗
Sedenka et al. [16]	HE+GC	ED+MD	✗	✓	✗	N/A	✗	✗
Boddeti et al. [6]	HE	CosSim	✓	✗	✗	✗	✗	✗
Im et al. [3]	HE	ED	✗	✓	✗	N/A	✗	✗
Engelsma et al. [4]	HE	CosSim	✓	✗	✗	✓	✗	✗
Osorio et al. [13]	HE	ED	✓	✗	✗	✗	✗	✗
Bauspieß et al. [17]	HE	ED+HD	✓	✗	✓	✗	✗	✗
Three-party Models								
Gasti et al. [1]	GC	HD+MD	✗	✓	N/A	N/A	✗	✗
Gunasinghe et al. [2]	ZKPK	SVM	✗	✓	N/A	N/A	N/A	✗
Hu et al. [12]	HE	ED	✓	✓	✗	✓	✗	✗
Drozdowski et al. [7]	HE	ED	✗	✓	✗	✗	✗	✗
Huang et al. [5]	HE+GC	ED	✓	✓	✗	✗	✗	✗
Huang et al. [8]	HE+GC	ED	✓	✓	✗	✓	✗	✗
BioFast (Proposed)	HE+Hash	CosSim	✓	✓	✓	✓	✓	✓

recognition model with a triple loss function called FaceNet. It can embed a face image into a feature vector and it is widely used. To obtain more discriminative features, the work [19] proposed a supervised signal called the central loss function for deep face recognition tasks. Liu et al. [20] proposed a face recognition model called SphereFace that can apply constraints to the hypersphere with an angular softmax loss function. The work of [21] and [22] proposed face recognition models, where the former focuses on maximizing the classification boundary in angular space while the latter maximizes the loss function of classification boundaries in cosine space.

Although the above face recognition approaches have relatively high recognition accuracy and recognition efficiency, they have not explored face privacy. Below, we recap privacy-preserving face recognitions.

A. Homomorphic Encryption-Based Recognition

To protect facial information, the sequential works [3], [4], [13], [15], [16], [17], [23], [24], [25], [26], [27], [28] studied homomorphic encryption based approaches in the two-party (client-server) model. To compute the similarity between the probe face feature and the stored template face feature, different types of distance functions are applied. The works of [3], [4], [13], [16], [17], [23], [24], [25], [26], [27], [28], [29] studied homomorphic encryption based face recognition in two-party model. Among them, the works [3], [4], [17], [23], [24], [25] applied the Euclidean distance in the similarity comparison process, where the works [4], [17], [23] studied the face identifications while the works [3], [24], [25] studied the face verification. The work of [4] used cosine similarity. It enhances the speed of encrypted biometric template identifications and achieves small storage in biometric templates. However, it doesn't consider the

privacy of similarity-threshold comparison. The server can get the similarity and cause a privacy risk.

The works of [1], [2], [5], [7], [8], [30] studied homomorphic encryption-based face recognition in three-party (client-server-CSP) model, where the work [7] applied Euclidean distance and the work of [29], [31] used Hamming distance. Ma et al. [31] proposed a privacy-preserving face verification scheme in a three-party model. It uses Hamming distance in similarity computation and OT in similarity-threshold comparison. The client can get the verification result from OT without knowing the similarity of the authentication. However, the CSP can get information on similarity in their method. Kolberg et al. [30] also studied the Hamming distance in similarity computation in the three-party model. They proposed three kinds of HE schemes for float, integer, and binary templates and achieved a small template storage by binary encryption method. They use an authentication server as a trusted third party (TTP) for similarity-threshold comparison. However, the TTP is not secure in the real world. Gasti et al. [1] used Hamming distance in a three-party model combined with the garbled circuits. Gunasinghe et al. The work of [2] studied face identification using SVM with the zero-knowledge proof technique. The works [5], [8] used Euclidean distance and homomorphic encryptions when computing the similarity, and used the garbled circuit when computing the threshold comparison. The work of [12] used the column method to store the feature vectors and used DGK protocol [11] to compare the similarity, and thus needs more interactions.

B. Others

In addition to using homomorphic encryptions in the privacy-preserving face recognition protocols, many other cryptographic

techniques are also applied to reach various security and efficacy goals. Gasti et al. [1] proposed an energy-efficient biometric authentication scheme for smartphones. This scenario securely outsources computations to an untrusted cloud. They conducted extended experiments to prove that their energy-saving solution could be used in smartphones. Sedenka et al. [16] proposed a secure outsourced privacy protection scheme for smartphone biometric authentication. However, the above-mentioned technology of protecting face recognition based on garbled circuits has a large communication overhead. Gunasinghe et al. [2] proposed a biometric verification scheme based on zero-knowledge proofs. Lagendijk et al. The work [27] proposed a privacy protection scheme for face recognition combining homomorphic encryption and garbled circuits. Huang Hai [5] proposed an efficient privacy face verification scheme based on homomorphic encryption and garbled circuits.

Ji et al. [32] proposed a privacy protection scheme for face recognition based on differential privacy. The scheme hides the face image information by removing the direct component called DC in the image frequency domain and constructs a learnable differential privacy budget method to allocate the privacy budget. Zhou et al. [25] proposed a user-centered biometric authentication scheme based on threshold predicate encryption. Bassit et al. [28] proposed a biometric verification scheme based on threshold homomorphic encryption and zero-knowledge proofs to defend against malicious attackers. The work of [33] proposed a cancelable multi-biometric template protection scheme that combines deep hashing with cancelable distance-preserving encryption (CDPE). Lei et al. [34] proposed a privacy-preserving face authentication, named present PRIV-FACE, supporting revocable, and reusable biometric credentials. To outsource the matrix operations (inversion, multiplications, and eigenvalue decomposition), the work of [35] designed a protocol that can outsource LDA-based face recognition to an untrusted cloud. Note that the comparison process also can be done with the Private Set Intersection (PSI) protocols [36]. However, the process needs homomorphic encryptions or more interactions. Although the work of [37] presented a very efficient and GPU-friendly comparison protocol, their method still relies on the DGK protocol and needs interaction. The work of [38] systematically analyzed the different types of combinations of the garbled circuits and homomorphic encryptions to private decision tree evaluation. Although the homomorphic or garbled circuit-based approaches are used to make the comparison, As we show in the last section, our hash-based approach achieves better efficiency with plausible space cost.

III. PRELIMINARIES

A. Feature Extraction Models and Distances

a) *Feature Extraction*: On inputting a biometric image, the feature extraction algorithm outputs a feature vector as Fig. 2. We can use this feature vector to perform biometric verification. For a face image, the well-known feature extraction model ArcFace can extract 512-dimensional face feature vectors from

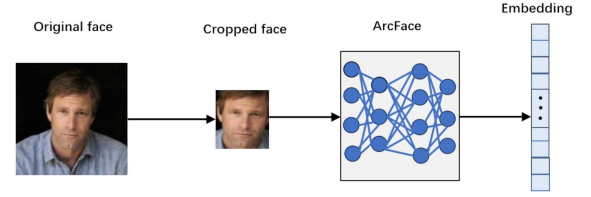


Fig. 2. The feature extracting process.

the face images. Each element of the vector is a floating-point number. For a fingerprint image, the well-known feature extraction algorithm FingerCode can extract 640 dimension vectors from fingerprint images. Each value of the vector is 256 values and requires 1-byte storage.

b) *Distances*: To evaluate the similarity between two face images, mathematical distances are used. Among them, the Euclidean distance, cosine distance, and Manhattan distance are common. The Euclidean distance and Manhattan distances can be defined via the vector norms. For a n -dimensional vectors $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, the p -norm is defined as

$$\|\mathbf{x}\|_p := \left(\sum_{i=1}^n x_i^p \right)^{1/p}.$$

In particular, for two vectors $\mathbf{x}$ and $\mathbf{y}$, the Euclidean distance of these two vectors corresponds to the 2-norm of the vector $\mathbf{x} - \mathbf{y}$. The Manhattan distance is corresponding to the vector 1-norm. The cosine similarity (distance) is also very common in the literates, which is defined as follows:

$$\text{CosSim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \cdot \mathbf{b}}{\|\mathbf{a}\| * \|\mathbf{b}\|},$$

where $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_n)$ are vectors over $\mathbb{R}^n$. Note that if the corresponding vectors $\mathbf{a}$ and $\mathbf{b}$ are unit vectors, then the cosine similarity turns into the inner product $\mathbf{a}^\top \cdot \mathbf{b}$.

B. Cryptographic Tools

a) *Fully Homomorphic Encryption (FHE)*: To compute on encrypted plaintext, the concept of FHE is proposed. Utilizing FHE schemes, we can perform additions and multiplications of plaintexts without decrypting the corresponding ciphertexts. More precisely, for the encryption algorithm, denoted Enc , the FHE scheme holds:

$$\text{Enc}(m_1) \oplus \text{Enc}(m_2) = \text{Enc}(m_1 + m_2)$$

$$\text{Enc}(m_1) \otimes \text{Enc}(m_2) = \text{Enc}(m_1 * m_2),$$

where m_1 and m_2 are messages; $\oplus$ and $\otimes$ are ciphertext operations in FHE scheme.

In privacy-protected biometric authentication, the FHE schemes are used to homomorphically compute the distance between the client's input feature vector and the server's stored feature vector. Since this computation process is performed on the ciphertexts of the feature vectors, privacy is protected.

b) *Rings in the Lattice FHEs*: Due to the efficiency and well-known security analysis, lattice-based homomorphic encryption schemes have become mainstream in the design and

applications of FHEs. Among the lattice-based FHE schemes, the BGV/BFV scheme is used to homomorphically compute the integers, and thus the integer vectors. Note that the BGV/BFV scheme is based on the quotient ring $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$ of the ring $\mathcal{R} := \mathbb{Z}[x]/(x^n + 1)$, where n is a power of 2, and q is some integer. A BGV/BFV ciphertext of a ring element in $\mathcal{R}_q$ consists of two ring elements in $\mathcal{R}_q$, where we call the modulus q the ciphertext modulus (also called the ring modulus) and q_p the plaintext modulus. The ciphertext additions and multiplications are performed over $\mathcal{R}_q$ using the modulus q , and the corresponding plaintext operations are held over $\mathcal{R}_{q_p}$. Usually, the plaintext modulus is much smaller than that of the ciphertext modulus. The ciphertext modulus strongly depends on the plaintext modulus and the multiplication depth. More details can be found in the work of [9], [10], [39].

A ring element in $\mathcal{R}$ can be represented as a coefficient vector in $\mathbb{Z}^n$, and we denote this representation map as *Coeffs*. Namely, for any ring element $a = \sum_{i=0}^{n-1} a_i x^i \in \mathcal{R}$, we have that $\text{Coeffs}(a) = (a_0, a_1, \dots, a_{n-1})^\top \in \mathbb{Z}^n$. Furthermore, we use another useful map ϕ , which maps a ring element in the ring $\mathcal{R}$ to a nega-cyclic matrix over $\mathbb{Z}^{n \times n}$ as

$$\begin{aligned} \phi(a) &:= \begin{bmatrix} a_0 & -a_{n-1} & \cdots & -a_1 \\ a_1 & a_0 & \cdots & -a_2 \\ \vdots & \ddots & & -a_{n-1} \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{bmatrix} \\ &= [\text{Coeffs}(a) | \text{Coeffs}(a \cdot x) | \cdots | \text{Coeffs}(a \cdot x^{n-1})]. \end{aligned}$$

For the elements in the quotient ring $\mathcal{R}_q$, the above maps run similarly.

c) *Hash Function*: On input of an arbitrary length message M , the hash functions output a fixed-length digest $H(M)$. This is known as the compression property of hash functions. In this paper, we use the cryptographic hash functions, which are more special kind of hash functions that possesses the following two more properties:

- *One-wayness*: Given the hash value $H(M)$, it is computationally hard to find the pre-image M .
- *Collisionresistence*: It is computationally hard to find two distinct messages M_1 and M_2 such that $H(M_1) = H(M_2)$.

There are constructions from computationally hard problems, e.g., discrete logarithm problems and shortest integer solution problems. Although the security of those constructions is based on well-known mathematical problems, because of their inefficiency, the direct designs [40] are more common in practical use.

IV. SYSTEM DESIGN AND METHODS

A. Overview of the System

The biometric recognition system mainly consists of two phases: the **Enrollment** phase and the **Verification** phase. The **Enrollment** phase is offline and can be carried out in the pre-processing stage, which will not bring additional time

overhead. The **Verification** phase is carried out online, and the time and communication overhead of the encryption protocol in the verification phase will directly affect the user's experience of using face verification. Note that, the extraction of face feature vectors also can be done with the feature extraction models, e.g., the ArcFace model, in the pre-processing stage. This extraction process can be independent of the **Enrollment** phase and the **Verification** phase, and thus it is not the main focus of this paper. Here, we mainly discuss the constructions of the **Enrollment** phase and the **Verification** phase. The high-level design of our privacy-preserving biometric verification system is given in Fig. 3, which mainly consists of the following two phases:

Enrollment : In this phase, the client requests the cryptographic parameters from the CSP. These parameters include the FHE parameters, public keys, and hash functions. Then, the client encrypts the feature vectors by the FHE public key and sends the ciphertext with the corresponding identities to the server. The server stores the ciphertext to further biometric identification.

Verification : To perform a verification, the client sends an encryption of the face feature vector v_{id} and the corresponding identity id to the server. After receiving the encrypted feature and the identity, the server homomorphically computes the similarity between the locally stored feature vector v'_{id} and the received feature vector v_{id} of the corresponding identity id . Then, the server cooperates with the CSP to securely decide if the similarity is smaller than the threshold, and send the final masked result to the client. Then, the client performs a local computation to get the verification result.

Note that, the **Verification** phase can be divided into two steps: (1) computing the distance (similarity) and (2) securely comparing the distance with the threshold. In the first step, the similarity is calculated homomorphically. In other words, if the Euclidean distance is used to compute the similarity of two biometric features, then the server will perform a homomorphic evaluation of 2-norm of the vector $(v_{id} - v'_{id})$. If the Manhattan distance is considered, then the server will perform the homomorphic evaluation of 1-norm of the vector $(v_{id} - v'_{id})$. To outperform the communication overheads, the existing protocols use the batching technique of FHEs. Yet, due to the heavy structure of the lattice-based FHEs, the communication overhead is still one of the most inefficient parts. In the next section, we introduce a more efficient feature embedding technique and thus obtain a more efficient protocol.

At the second step, to prevent the client from obtaining further information beyond the final identification result, the CSP is introduced, which has the secret key of the FHE ciphertexts. The CSP will assist the server in completing the task of comparing the feature vectors' distance with the threshold. To prevent the CSP from getting the similarity distance d , the server uses the random number r to mask the distance d . By now, the server and the CSP have an additive share of the distance d , then the secure computation technique, e.g., Garbled circuits and Beaver's triple, can be used to further compute the comparison. This would bring a relatively large amount of interactions,

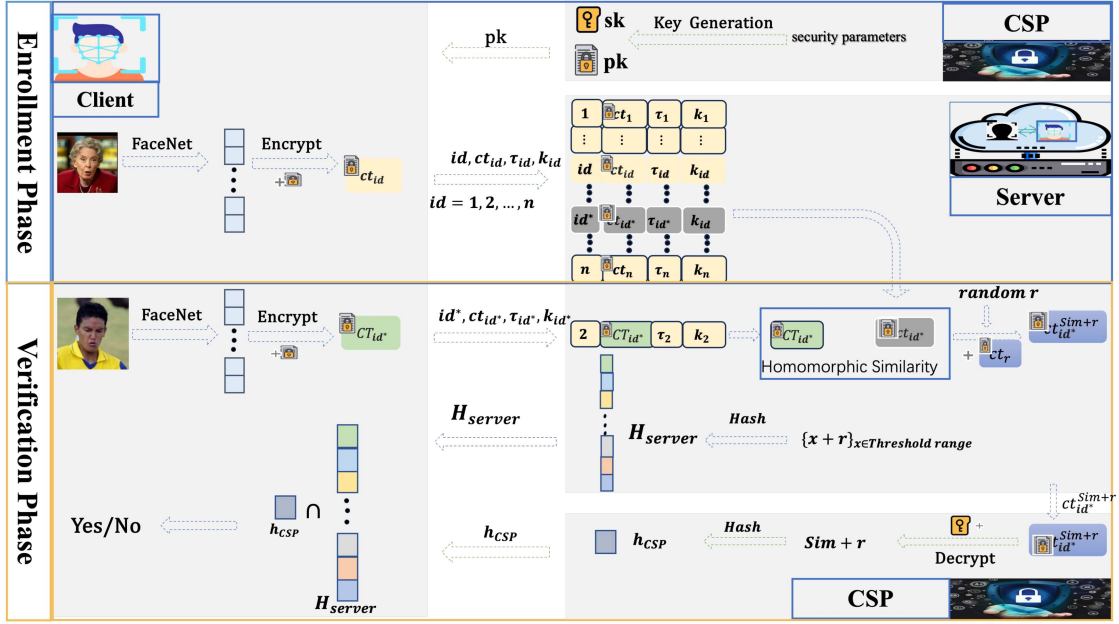


Fig. 3. The high-level overview of our face recognition system.

which would cause a relatively large time overhead in the high latency network. This motivates us to search for a solution with smaller communication and time overhead. We observe that the multiparty comparison can be efficiently accomplished via the cryptographic hash evaluations if the threshold is small enough, and thus, we obtain a verification system that achieves smaller communication and time overhead.

B. More Compact Similarity Computation

In this section, we show a detailed description of our techniques of embedding a feature vector into a ring element in $\mathcal{R}$, thus getting a more efficient computation of cosine similarity. Before showing our embedding technique, below, we present a useful observation in the polynomial multiplication in the ring $\mathcal{R}$.

The multiplication of two polynomials can be accomplished by the multiplication of the nega-cyclic matrix corresponding to the first ring polynomial by the coefficient vector corresponding to the second ring polynomial. More precisely, for the two ring elements $a \in \mathcal{R}$ and $b \in \mathcal{R}$, we have

$$\begin{aligned} \text{Coeffs}(a \cdot b) &= \begin{bmatrix} a_0 & -a_{n-1} & \cdots & -a_1 \\ a_1 & a_0 & \cdots & -a_2 \\ \vdots & \ddots & & -a_{n-1} \\ a_{n-1} & a_{n-2} & \cdots & a_0 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{n-1} \end{bmatrix} \\ &= \phi(a) \cdot \text{Coeffs}(b), \end{aligned} \quad (1)$$

where the maps ϕ and Coeffs are defined in the previous section. In addition to the above embedding maps, we define a map from integer vectors to the ring $\mathcal{R}$ as follows.

Definition 1: For the ring $\mathcal{R}$ as defined in the preliminary, a positive integer $k \leq n$, and a vector $v = (v_0, v_1, \dots, v_{k-1}) \in \mathbb{Z}^k$, the embedding map $\text{Embedd}_j : \mathbb{Z}^k \rightarrow \mathcal{R}$ indexed by some

$j < \lfloor \frac{n}{k} \rfloor$ is defined as follows:

$$\text{Embedd}_j(v) = - \sum_{i=0}^{k-1} v_i \cdot x^{n-jk-i}.$$

Furthermore, we note that the cosine similarity of the two unit vectors v and v' is computed by simple inner product, that is $\text{CosSim}(v, v') = v^\top \cdot v'$. Let Coeffs^{-1} be the inverse map of the coefficient embedding map Coeffs , we have the following result from the above observations.

Theorem IV.1: Let v and v' be two unit vectors, then the cosine similarity $\text{CosSim}(v, v')$ equals to the constant term of the polynomial $\text{Embedd}_0(v) \cdot \text{Coeffs}^{-1}(v')$.

Proof: From the equation (1), we know that

$$\begin{aligned} &\text{Coeffs}(\text{Embedd}_0(v) \cdot \text{Coeffs}^{-1}(v')) \\ &= \phi(\text{Embedd}_0(v)) \cdot \text{Coeffs}(\text{Coeffs}^{-1}(v')) \\ &= \phi(\text{Embedd}_0(v)) \cdot v'. \end{aligned}$$

From the Definition 1, we further have

$$\begin{aligned} \phi(\text{Embedd}_0(v)) \cdot v' &= \phi\left(- \sum_{i=0}^{k-1} v_i \cdot x^{n-i}\right) \cdot v' \\ &= \begin{bmatrix} v_0 & v_1 & \cdots & v_{k-1} & 0 & \cdots & 0 \\ 0 & v_0 & v_1 & \cdots & v_{k-1} & \cdots & 0 \\ \vdots & & & \ddots & & & \\ -v_2 & \cdots & -v_{k-1} & & & v_0 & v_1 \\ -v_1 & \cdots & & -v_{k-1} & \cdots & 0 & v_0 \end{bmatrix} \cdot \begin{bmatrix} v'_0 \\ v'_1 \\ \vdots \\ v'_{k-1} \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \end{aligned}$$

Algorithm 1: BatchInnerProduct($\{v_i, v'_i\}_{i \in [\tau]} \rightarrow \{v_i^\top \cdot v'_i\}_{i \in [\tau]}$).

Input: $\{v_i, v'_i \in \mathbb{Z}^k\}_{i \in [\tau]}$ $\triangleright$ Two sets of vectors
Output: $\{v_i^\top \cdot v'_i\}_{i \in [\tau]}$ or $\perp$ $\triangleright$ Set of inner products
1: Let $v' := [v_1^\top | \dots | v_\tau^\top]^\top$
2: Compute $b := \text{Coeffs}^{-1}(v')$
3: **if** $k > n$ or $\tau > \lfloor \sqrt{\frac{n}{k}} \rfloor$ **then**
4: Output $\perp$
5: **end if**
6: Set $a = \sum_{\kappa_a=1}^\tau \text{Embedd}_{(\kappa_a-1)\tau}(v_{\kappa_a})$
7: Compute $c = a \cdot b \in \mathbb{R}$
8: **return** $\{c_{n-(\tau-1)\kappa_a k}\}_{\kappa_a \in \{0,1,\dots,\tau-1\}}$ $\triangleright$ Outputs coefficients of c

Thus, the constant term $\text{Embedd}_0(v) \cdot \text{Coeffs}^{-1}(v')$ corresponds to the multiplication of the first row of the above negacyclic matrix and the vector v' , that is $\sum_{i=0}^{k-1} v_i \cdot v'_i$. Since v and v' are unit vectors, we have that $\text{CosSim}(v, v') = \sum_{i=0}^{k-1} v_i \cdot v'_i$, and thus the theorem follows. ■

It is not hard to observe from the above theorem that the homomorphic evaluation of the cosine similarity can be achieved via homomorphically multiplying corresponding coefficient-embedded ring polynomials. In other words, we are capable of homomorphically computing the similarity between the two encrypted feature vectors. Additionally, our embedding is coefficient-wise, thus any length $k \leq n$ vector can be embedded into one ring vector, and thus via one homomorphic polynomial multiplication, that is $\text{Enc}(\text{Embedd}_0(v)) \otimes \text{Enc}(\text{Coeffs}^{-1}(v'))$, we can homomorphically compute the cosine similarity.

Note that the biometric features are usually 512-dimensional vectors while the ring dimension n is at least 1024 to provide homomorphic evaluations. To provide homomorphic multiplication, the ring element may grow to 4096 or larger (up to the message size). Thus, via the above embedding, many coefficients are set to 0, and not utilized. Below, we show how to pack more vectors into one ring element and compute the cosine similarity simultaneously.

For two column vectors $v \in \mathbb{Z}^k$ and $v' \in \mathbb{Z}^k$, Let $[v^\top | v'^\top]^\top \in \mathbb{Z}^{2k}$ be the concatenation of them, and let $[\tau]$ be the index set $\{1, 2, \dots, \tau\}$, then the multi-vector embedding and the batch computing of corresponding cosine similarity is presented in the following Algorithm 1.

Theorem IV.2: Let n be the ring dimension, $k < n$ be the dimension of the vectors $\{v_i, v'_i\}_{i \in [\tau]}$ for some integer $\tau^2 \leq \lfloor \frac{n}{k} \rfloor$, then the Algorithm 1 outputs the set of inner products $\{v_i^\top \cdot v'_i\}_{i \in [\tau]}$.

Remark IV.3: The above theorem implies that we can pack τ independent feature vectors (of dimension k) into a single ciphertext (in the ring of dimension n) as long as $\tau^2 < \lfloor n/k \rfloor$. Thus, for a fixed ring dimension n , we can pack multiple (at least 2) independent feature vectors whenever $n \geq 4k$. Otherwise, the method can't pack multiple feature vectors, and the method degrades to the plain method.

Note that if the vectors are normalized, then the inner product is equal to the cosine similarity, and thus, applying the above algorithm, we can compute more similarity simultaneously. This technique doesn't even affect the space and time efficiency at all. As discussed above, if the ring dimension is 4096, and a 512-dimensional biometric feature vector is used, then we can embed two features into one ring element, and the computation of cosine similarity is obtained via simply extracting the corresponding coefficients.

In addition, to compute the similarity, the biometric verification process needs to compare the similarity value with a threshold value t . Next, we show how to use cryptographic hash functions to compare these two values.

C. More Efficient Similarity-Threshold Comparison

In the biometric similarity-threshold comparison, the threshold generally is a small positive. Let d be the computed similarity, and t be the threshold, then $d \leq t$ is equivalent to the case $d \in [0, t]$. Prior to showing our similarity-threshold comparison, we present a more general case of deciding if an integer is in the given interval. Below, we show our observations and related results.

For a similarity d and an interval $[B_l, B_u]$, it is not hard to see that the following probability holds for any real number $r \in \mathbb{R}$.

$$\Pr[d + r \in [B_l, B_u] + r] = 1 \text{ if } d \in [B_l, B_u],$$

where $[B_l, B_u] + r$ denotes the interval $[B_l + r, B_u + r]$.

We observe that if an integer d belongs to an interval, then there is an integer d_0 in the interval such that $d = d_0$. This works well, yet the efficiency is a real concern. Nonetheless, if the interval is small, this exhaustive equality testing process runs extremely fast. The crux here is that the number of equality tests will grow large if the interval is large. We observe that by shrinking the size of the interval and the number at hand by a factor of p and then rounding them to an integer, the comparisons become more efficient, yet this may introduce some error in the accuracy result. The following theorem captures the corresponding probability.

Below, we use $[B_l, B_u]_{\mathbb{Z}}$ to denote the set of integers in the set $[B_l, B_u]$. Since we work on the quotient ring $\mathbb{Z}_{q_p}$ during the non-interactive comparisons, therefore, W.l.o.g, we still use $[B_l, B_u]_{\mathbb{Z}}$ to denote the wrapped interval. Since d represents the amplified cosine similarity, it is upper bounded by some integer B_u . Below, we just analysis this case. For the simplicity of the description, we made following notations: $B_{q_p, p}^{\ell, r} := \lfloor \frac{B_l + r \bmod q_p}{p} \rfloor$, $B_p^{\ell, r} := \lfloor \frac{B_l + r}{p} \rfloor$, $B_{q_p, p}^{u, r} := \lfloor \frac{B_u + r \bmod q_p}{p} \rfloor$, $B_p^{u, r} := \lfloor \frac{B_u + r}{p} \rfloor$, $d_{q_p, p}^r := \lfloor \frac{d + r \bmod q_p}{p} \rfloor$, $d_p^r := \lfloor \frac{d + r}{p} \rfloor$. Below we evaluate the border effect of $d_{q_p, p}^r$.

Theorem IV.4: Let p be a positive integer, $[B_l, B_u]$ be an interval such that $B_u - B_l \geq p$, q_p be a positive integers such that $p|q_p$ and $q_p > (B_u - B_l) + 2p$, and r be a random integer over $\mathbb{Z}_{q_p}$, then we have following results for an integer $d < B_u$:

- FPR property: If $d \in [B_\ell - p, B_\ell]_{\mathbb{Z}}$, then

$$\begin{aligned} \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \right] \\ = \left(1 - \frac{B_\ell - d}{p} \right) \cdot \frac{q_p - B_\ell + d}{q_p}. \end{aligned}$$

Proof: From the law of total probability, we have

$$\begin{aligned} \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \right] \\ = \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid d + r \geq q_p \right] \cdot \Pr[d + r \geq q_p] \\ + \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid d + r < q_p \right] \cdot \Pr \left[d + r < q_p \right] \\ + \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_\ell + r \geq q_p \right] \cdot \Pr \left[B_\ell + r \geq q_p \right] \\ + \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_\ell + r < q_p \right] \cdot \Pr \left[B_\ell + r < q_p \right] \\ + \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_u + r \geq q_p \right] \cdot \Pr \left[B_u + r \geq q_p \right] \\ + \Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_u + r < q_p \right] \cdot \Pr[B_u + r < q_p], \end{aligned}$$

where, from the randomness of r over $\mathbb{Z}_{q_p}$, we have

$$\Pr[d + r \geq q_p] = \frac{d}{q_p}, \quad \Pr \left[\begin{matrix} d + r < q_p \\ B_\ell + r \geq q_p \end{matrix} \right] = \frac{B_\ell - d}{q_p},$$

$$\Pr \left[\begin{matrix} B_\ell + r < q_p \\ B_u + r \geq q_p \end{matrix} \right] = \frac{B_u - B_\ell}{q_p}, \quad \Pr[B_u + r < q_p] = \frac{q_p - B_u}{q_p}.$$

Next, we compute the condition probabilities. If $d + r \geq q_p$, then we have

$$d_{q_p, p}^r = d_p^r - q_p/p, B_{q_p, p}^{\ell, r} = B_p^{\ell, r} - q_p/p, B_{q_p, p}^{u, r} = B_p^{u, r} - q_p/p.$$

Thus, $d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}}$ is equivalent to $d_p^r \in [B_p^{\ell, r}, B_p^{u, r}]_{\mathbb{Z}}$. Therefore, we have

$$\Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid d + r \geq q_p \right] = 1 - \frac{B_\ell - d}{p}.$$

Similarly, we have

$$\Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_u + r < q_p \right] = 1 - \frac{B_\ell - d}{p}.$$

From the theorem assumption that $q_p > (B_u - B_\ell) + 2p$, we have $B_u + r - q_p < B_\ell + r - 2p < d + r - p$, and thus $B_{q_p, p}^{u, r} \leq d_p^r - 1$. If $B_\ell + r < q_p$ and $B_u + r \geq q_p$, then $d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}}$ (wrapped interval) is equivalent to $d_p^r \in [B_p^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}}$. Thus, we have

$$\Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid B_u + r \geq q_p \right] = 1 - \frac{B_\ell - d}{p}.$$

In the case of $B_\ell + r \geq q_p$ and $d + r < q_p$, from the above analysis we have that $d_{q_p, p}^r \notin [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}}$, and thus

$$\Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \mid d + r < q_p \right] = 0.$$

Put all the above results together, we have that

$$\Pr \left[d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]_{\mathbb{Z}} \right] = \left(1 - \frac{B_\ell - d}{p} \right) \cdot \frac{q_p - B_\ell + d}{q_p}$$

The above theorem only characterizes the false positive rates of the modulus rounding and scaled operations. Intuitively, these operations may also impact on the false negative rate. However, whenever $d \in [B_\ell, B_u]$, d will not be rejected. This is the case, since $d \in [B_\ell, B_u]$, we have that $B_\ell + r \leq d + r \leq B_u + r$. If $d + r < q_p$, then $d_{q_p, p}^r \in [B_{q_p, p}^{\ell, r}, q_p/p]$; If $d + r \geq q_p$, then $d_{q_p, p}^r \in [0, B_{q_p, p}^{u, r}]$. Thus, whichever the case, $d_{q_p, p}^r$ always in the wrapped interval $[B_{q_p, p}^{\ell, r}, B_{q_p, p}^{u, r}]$. Therefore, these operation will not introduce new false negative rate for properly selected parameters such that $p|q_p$. ■

The above theorem shows that the integer $\left\lfloor \frac{d+r}{p} \right\rfloor$ may fall in to the integer interval $\left[\left\lfloor \frac{B_\ell+r}{p} \right\rfloor, \left\lfloor \frac{B_u+r}{p} \right\rfloor \right]_{\mathbb{Z}}$ albeit $d \notin [B_\ell, B_u]_{\mathbb{Z}}$. The probability of corresponding error is determined by distance $B_\ell - d$ (or $d - B_u$) and the shrinking factor p . Note that the larger the factor p is, the more likely the error happen, and vice versa. However, the smaller factor p results in a larger interval, which decreases the efficiency of the comparison process. By appropriately selecting the factor p , we can achieve better efficiency and quite plausible accuracy at the same time. Concrete performance evaluation is shown in the next section.

V. DETAILED DESIGN OF OUR PROTOCOL

Applying the techniques presented in the previous sections, we next present the design of our biometric recognition protocol in detail. The design process is shown in the following Fig. 4. In the following sections, we show a detailed description of each part.

A. Biometric Enrollment

As shown in Fig. 4, the biometric enrollment process is accomplished under the cooperation of three parties: the client, the server, and the CSP. Each party operates as follows:

CSP: On receiving the homomorphic public key request from a client, it sends the corresponding BFV public key pk to the client.

Client: To enroll a biometric identity id , the client first extracts the k_{id} -dimensional unit feature vector $\bar{f}_{id}$ from the biometric image through the feature extraction model, e.g., FaceNet. Then, it enlarges the feature vector by factor of γ , that is $f_{id} := \lfloor \gamma \cdot \bar{f}_{id} \rfloor$, and then encrypts the enlarged feature vector f_{id} via the public key pk of the BFV scheme, which comes from the CSP. Namely, the client computes the ciphertext $ct_{id} = \text{Enc}_{pk}(\text{Coeffs}^{-1}(f_{id}))$ under the BFV public key pk , and then sends the tuple $(id, ct_{id}, \tau_{id}, k_{id})$ to the server, where τ_{id} is the batch size chosen by the client.

Server: On receiving an identity-ciphertext pair $(id, ct_{id}, \tau_{id}, k_{id})$ from the client, the server runs the algorithm $\text{Enroll}((id, ct_{id}, \tau_{id}, k_{id}), ID, CT, \tau_D, k_D)$ as shown in the Algorithm 2, where ID is the set of enrolled identities, k_D and τ_D are the feature dimension and batch size that the server side supports, and the CT is the set of ciphertexts of corresponding batched

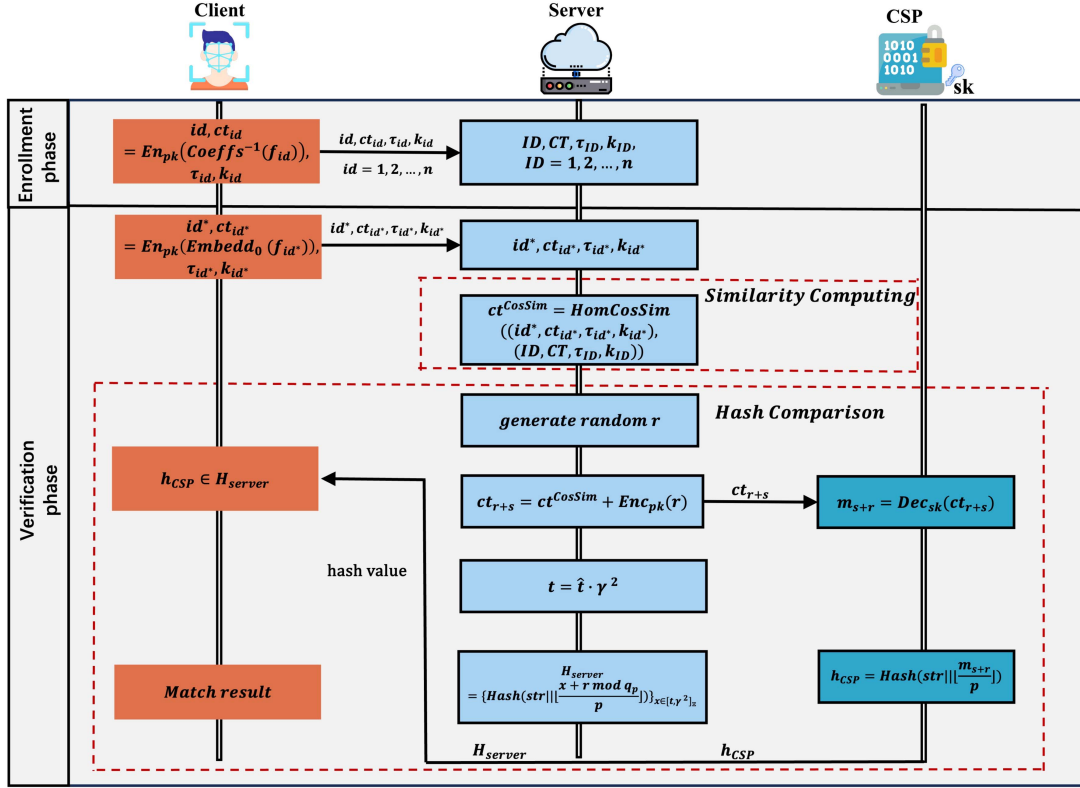


Fig. 4. The design process of our protocol.

Algorithm 2: Enroll((id, ct_{id}, τ_{id}, k_{id}), (ID, CT, τ_{ID}, k_{ID})).**Input:** id, ct_{id}, τ_{id}, k_{id} ID, τ_{ID}, k_{ID}, CT = {CT_i}_{i ∈ [⌊ $\frac{|ID|}{\tau_{ID}}$ ⌋]}**Output:** Success or Fail

- 1: **if** |ID| = 0 and k_{id} ≤ n **then**
- 2: Set k_{ID} := k_{id}, and τ_{ID} := ⌊ $\sqrt{\frac{n}{k_{ID}}}$ ⌋
- 3: **if** τ_{id} ≠ τ_{ID} or k_{id} ≠ k_{ID} **then**
- 4: Return Fail
- 5: Set ℓ := |ID| − τ_{ID} · ⌊ $\frac{|ID|}{\tau_{ID}}$ ⌋
- 6: **if** ℓ = 0 **then**
- 7: Set CT_{⌊ $\frac{|ID|}{\tau_{ID}}$ ⌋+1} = ct_{id}, and append id into ID}
- 8: Return Success
- 9: Compute ct'_{id} := ct_{id} ⊗ Enc_{pk}(x^{ℓ·τ_{ID}})
- 10: Compute CT_{⌊ $\frac{|ID|}{\tau_{ID}}$ ⌋} := CT_{⌊ $\frac{|ID|}{\tau_{ID}}$ ⌋}} + ct'_{id}}
- 11: Append id into ID, and Return Success

features. If the output of the algorithm Enroll is *Fail*, then the server sends an abort message to the client and declines the verification process; otherwise, continue the verification process.

Let ID be the set of identities, the enrolled location (or simply location, and denoted L_{id}) of an identity id in the ID indicates that the feature of the identity id is enrolled into the server's storage at L_{id} -th enrollment. It is not hard to see that each ciphertext CT_i packs τ ciphertexts of the corresponding features if $i \leq \lfloor \frac{|ID|}{\tau_{ID}} \rfloor$.

B. Biometric Verification

As shown in Fig. 4, the biometric verification process can be divided into two phases: similarity computation, and threshold comparison. Below, we show a concrete description of them.

a) *Similarity Computation:* To compute the biometric features similarity, the client and the server works as follows:

Client: To authenticate an identity id*, the client first obtains a k_{id*} dimensional enlarged biometric feature vector f_{id*} := ⌊γ · f_{id}⌋ from the extraction model, then embeds the (enlarged) feature vector into a ring polynomial, and then encrypts it with the BFV public key pk. Namely, the client computes ct_{id*} := Enc_{pk}(Embed₀(f_{id*})), and then sends the tuple (id*, k_{id*}, τ_{id*}, ct_{id*}) to the server, where k_{id*} is the dimension of feature vector, τ_{id*} is the batch size.

Server: On receiving the feature messages (id*, k_{id*}, τ_{id*}, ct_{id*}), the server runs the Algorithm 3 to homomorphically compute the cosine similarity. If the output of the algorithm is ⊥, then the server sends an abort message to the client and declines the verification process. If the output of the algorithm is a ciphertext, then the server continues the verification process.

It is not hard to see from the description of Algorithm 3 and Algorithm 2 that they are the homomorphic version of the batch inner product algorithm described (that used our packing technique) in Algorithm 1. Steps 1-2 of the Algorithm 1 correspond to the steps 9-10; Steps 5-6 of Algorithm 3 correspond to steps 5-6 of Algorithm 1. Other steps are used to ensure the packing works correctly. Note that if the algorithm HomCosSim outputs

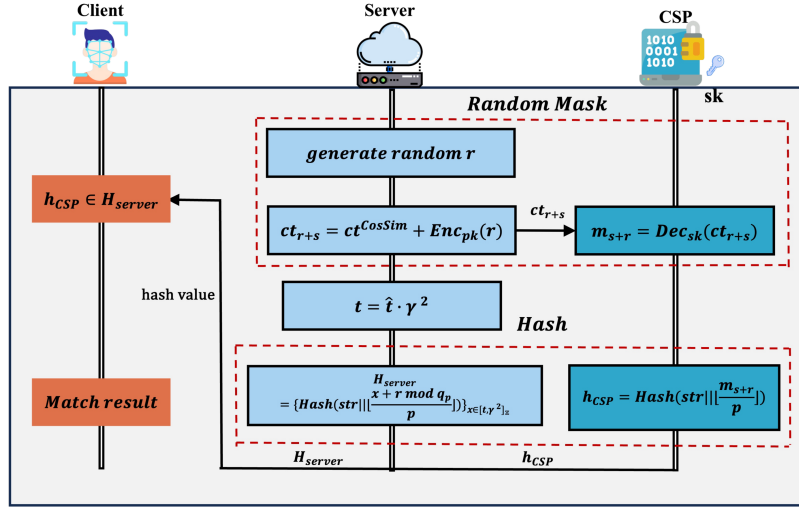


Fig. 5. The workflow of threshold-similarity comparison.

Algorithm 3: HomCosSim((id*, ct_{id*}, τ_{id*}, k_{id*}), (ID, CT, τ_{ID}, k_{ID})).

Input: id*, ct_{id*}, τ_{id*}, k_{id*}

ID, τ_{ID}, k_{ID}, CT = {CT_i}_{i ∈ [⌊ID/τ_{ID}⌋]}

Output: A ciphertext or ⊥

1: if τ_{id*} ≠ τ_{ID} or k_{id*} ≠ k_{ID} or id* ∈ ID then

2: Return ⊥

3: Let L_{id*} be the location of id* in ID

4: Set ℓ* := L_{id*} - ⌊L_{id*}/τ_{ID}⌋

5: Set ct'_{id*} := ct_{id*} ⊗ Enc_{pk}(x^{-(ℓ*-1)·τ_{ID}·k_{ID})}

6: Set ct^{CosSim} := ct'_{id*} ⊗ CT_{⌊L_{id*}/τ_{ID}⌋}}

7: Return ct^{CosSim}.

a ⊥ means that the batch size τ_{id*} is not match the server-side batch size τ_{ID} of the biometric feature dimension not match. If the final output is a ciphertext ct^{CosSim}, then the similarity between the biometric features from the client and the feature stored in the server should be computed in the corresponding coefficient of the decryption of ct^{CosSim}. The following theorem formally shows this fact. Due to the space limit, the proof of the theorem defer to full version.

Theorem V.1: For a registered template feature f of the identity id, and a prob feature f* corresponding to the same identity id, if the ciphertext ct^{CosSim} is returned by the Algorithm 3, then the (n - (τ_{ID} - 1) · ℓ* · k_{ID})-th coefficient of the decrypted message Dec_{sk}(ct^{CosSim}) is the inner product of the probe feature f* and template feature f.

Note that the encrypted features are enlarged by a factor of γ, thus the final inner product is enlarged by γ². After securely computing the enlarged similarity, the next goal is to compare this similarity with the threshold. Below, we show how to perform threshold comparison and then obtain the final verification result.

b) *Similarity-Threshold Comparison:* From the definition of the threshold t, for the the similarity s, we have s ∈ [t, γ²] if and

only if s ≥ t. Thus, the similarity-threshold comparison can be done on the client side by checking if the similarity is in the fixed set as Fig. 5. Since the similarity s can't be given to the client, the CSP, or the server, we use a randomness r to mask the similarity. Thus, the CSP now only gets the masked value r + s ∈ ℤ_{q_p}. Although s ∈ [t, γ²] holds if and only if r + s mod q_p is in the shifted wrapped interval [t + r mod q_p, γ² + r mod q_p]. Simply sending the interval to the client leaks the randomness r. Thus, via replacing the shifted values with the keyed hash values, the client can check if s ≥ t without knowing the similarity at all.

Note that the client and the server hold the public key pk of BFV encryption, and CSP holds the private key sk of BFV encryption. Let s be the (n - (τ_{ID} - 1) · ℓ* · k_{ID})-th coefficient of the ciphertext ct^{CosSim}, that is s := ct^{CosSim}_{n - (τ_{ID} - 1) · ℓ* · k_{ID}}, t̂ < 1 be the public similarity threshold, and p be some positive integer that is used to shrinking the interval, then each party runs as follows:

Server: The server randomly select a random element r, and sets the ring element $\mathbf{r} := r \cdot x^{n - (\tau_{ID} - 1) \cdot \ell^* \cdot k_{ID}} \in \mathcal{R}_{q_p}$, and then sends ciphertext $ct_{r+s} := ct^{CosSim} + Enc_{pk}(\mathbf{r})$ and an coefficient index $n - (\tau_{ID} - 1) \cdot \ell^* \cdot k_{ID}$ to the CSP. In addition, the server generates a random 256-bit string str, and then sends str to the CSP. Next, the server enlarges the threshold t̂ by a factor of γ² to compute $t := \hat{t} \cdot \gamma^2$. Next, computes the set of hashes $H_{server} := \{h_x := Hash(str || \lfloor \frac{x+r \bmod q_p}{p} \rfloor)\}_{x \in [t, \gamma^2]_{\mathbb{Z}}}$, and sends the shuffled hashes to the client.

CSP: After receiving the ciphertext ct_{r+s} from the server, the CSP decrypts it to get the n - (τ_{ID} - 1) · ℓ* · k_{ID}-th coefficient, denoted m_{s+r} ∈ ℤ_{q_p}. Thereafter it computes the hash value $h_{CSP} := Hash(str || \lfloor \frac{m_{s+r}}{p} \rfloor)$ for the shrinking factor p, and then sends h_{CSP} to the client.

Client: After receiving the set hashes H_{server} from the server, and a hash h_{CSP} from the CSP, the client simply checks if the hash value h_{CSP} is in the set of hashes H_{server}. If the

result is true, then it means that the biometric verification is successful, and the verification fails otherwise.

Since the CSP has the masked similarity $r + s$ while the server has the randomness r , the case that the client gets a successful verification by checking $h_{CSP} \in H_{server}$ implies that

$$\left\lfloor \frac{s + \text{mod } q_p}{p} \right\rfloor \in \left[\left\lfloor \frac{t + \text{mod } q_p}{p} \right\rfloor, \left\lfloor \frac{\gamma^2 + \text{mod } q_p}{p} \right\rfloor \right]_{\mathbb{Z}} \quad (2)$$

From the Theorem IV.4, we know that, for some $s < t$ and thus $s \notin [t, \gamma^2]$, the value $\left\lfloor \frac{s + \text{mod } q_p}{p} \right\rfloor$ may belong to the wrapped interval $\left[\left\lfloor \frac{t + \text{mod } q_p}{p} \right\rfloor, \left\lfloor \frac{\gamma^2 + \text{mod } q_p}{p} \right\rfloor \right]_{\mathbb{Z}}$. Therefore, the above equation doesn't directly imply $s < t$. Yet, by setting the factor p properly, we get an extremely fast threshold comparison scheme with almost no loss in the accuracy of the verification result. The concrete experimental results will be presented in the following section.

As per the above discussion, if the similarity s is near the threshold t , then the verification may output success with a very small probability. If the similarity s is far from the threshold t , then it is computationally impossible for the verification process to output a success. From the collision resistance of the cryptographic hash function, the different values map to different values, and thus the event $h_{CSP} \in H_{server}$ couldn't happen.

Remark V.2: We remark that the evaluation process of the hash function needs the secret random string str , which is only known to the CSP and the server. This can prevent the client from exhaustively guessing the preimages of the hash values. Thus, the client only able to decide whether the hash value h_{CSP} from CSP is in the set H_{server} from the server. Note that introducing such secret string negligibly impacts the communication or the computation overhead of the protocol.

VI. THEORETICAL ANALYSIS

In this section, we analyze our protocol in both the theoretical security and theoretical performance.

A. Security Analysis

Here in this section, we present the security assumptions, and security goals that our biometric verification protocol can achieve.

1) Security Model: Since the SSL protocols are widely used in real-world scenarios, we assume that the communication tunnel among the participants is protected by the SSL protocols. Thus, any communication tunnel eavesdropper is no longer a threat to the verification protocol. In what follows, we assume that the client, server, and CSP are all semi-honest; There is no collusion between any two parties of them. During the verification process, the main adversarial goal of each party is as follows:

- The client is curious about the feature templates stored on the server side and the corresponding exact similarity distances and tries to extract more information except for the final verification result.

- The server is curious about the client's probe feature, the template features stored at the server, and the corresponding exact similarity distances.
- The CSP is curious about the client's probe feature, the feature templates stored on the server side, and the corresponding exact similarity Distances.

Below, we show that our protocol can protect all semi-honest behaviors.

2) Security Analysis: To show the security of the proposed protocol, we use simulation-based techniques. Formally, for a protocol Π and a semi-honest participant $\mathcal{A}$, the protocol's security is defined via the real game and the simulated (or ideal) game. In the real game, the transactions received by the semi-honest participant are generated by real protocols. However, in the ideal game, these transactions are generated by a simulator that only has the output of the ideal functionality and these public parameters. Let $\text{View}_{\mathcal{A}}^{\Pi}$ be the view of the $\mathcal{A}$ in the real game, and $\text{View}_{\mathcal{A}}^{\text{Sim}}$ be the view of the $\mathcal{A}$ in the simulated ideal game.

Definition 2: we call a face identification protocol Π is secure against a semi-honest participant P if, for all PPT participant $\mathcal{A}$, there exists a PPT simulator Sim such that

$$\text{View}_{\mathcal{A}}^{\Pi} \stackrel{c}{\approx} \text{View}_{\mathcal{A}}^{\text{Sim}}$$

If the simu has extra information $f(x)$ on the input x , then we call the protocol secure with the leakage of $f(x)$.

Note that if the simulator has the extra information $f(x)$, then any semi-honest participant can have, at most, the information $f(x)$. Below, we show the security of the proposed protocol against all the semi-honest participants. For the sake of the simplicity of the proof, we let pk, sk be the public/secret key of the underlying FHE scheme; f^* is the probe face feature vector; H_{server} is the set of the hash values of the wrapped set $[t + r \bmod q_p, \gamma^2 + r \bmod q_p] \cap \mathbb{Z}$ for the threshold t and the amplification factor γ ; and h_{CSP} is the hash value of $s + r \bmod q_p$ for the random integer $r \in \mathbb{Z}_{q_p}$ and the similarity s . Then the security results are as follows.

Theorem VI.1: The proposed protocol Π is secure against semi-honest clients with leakage of the verification result in the random oracle model.

Proof: Note that the client receives the set of hash functions H_{server} from the server and receives the hash value h_{CSP} from the CSP. Therefore, the view of the client in the real game is $\text{View}_{client}^{\Pi} := (\text{pk}, f^*, H_{server}, h_{CSP})$, where $h_{CSP} \in H_{server}$ if (2) holds, and $h_{CSP} \notin H_{server}$ otherwise. The simulator Sim in the ideal game runs as follows:

$\text{Sim}(\text{pk}, b) \rightarrow (H_{server}, h_{CSP})$: On input a bit $b \in \{0, 1\}$ ($b = 1$ if the verification result is positive, and $b = 0$ otherwise) and the public parameters, the simulator samples $\gamma^2 - t$ random strings, and construct a set $H_{server} := \{H_{server,i}\}_{i \in [\gamma^2 - t]}$ each of length $|\text{Hash}(s + r \bmod q_p)|$. Then, it let $h'_{CSP} := H_{server,\tau}$ for some random $\tau \in [\gamma^2 - t]$ if $b = 1$, or set h'_{CSP} as a length $|\text{Hash}(s + r)|$ random string if $b = 0$. Finally, it sends (H_{server}, h_{CSP}) to the client. ■

Now, the view of the client in the ideal game is that $\text{View}_{client}^{\text{Sim}(b)} := (\text{pk}, f^*, H'_{server}, h'_{CSP})$. From the theorem assumption that

Hash is the random oracle, H_{server} is randomly distributed, which is the same as H'_{server} . In addition, the relation between h_{CSP} and H_{server} is the same as h'_{CSP} and H'_{server} . Therefore, the views of the semi-honest client in the real game and the simulated game are indistinguishable, that is, $\text{View}_{client}^{\Pi} \stackrel{c}{\approx} \text{View}_{client}^{\text{Sim}(b)}$. ■

Below, we show the security against the semi-honest server.

Theorem VI.2: The proposed protocol Π is secure against the semi-honest server if the underlying FHE is IND-CPA secure.

Proof: Note that the server receives the ciphertext ct_f , of the template feature f and the ciphertext ct_{f^*} of the probe feature f^* from the client. Therefore, the view of the server in the real game is $\text{View}_{server}^{\Pi} := (\text{pk}, r, \text{ct}_f, \text{ct}_{f^*})$, where r is a random integer over $\mathbb{Z}_q$. The simulator Sim in the ideal game runs as follows:

$\text{Sim}(\text{pk}) \rightarrow (\text{ct}'_f, \text{ct}'_{f^*})$: On input the public key of the FHE scheme, the simulator computes ct'_f and ct'_{f^*} as follows:

$$\text{ct}'_f := \text{FHE} \cdot \text{Enc}(0), \text{ct}'_{f^*} := \text{FHE} \cdot \text{Enc}(1).$$

Then, it sends $(\text{ct}'_f, \text{ct}'_{f^*})$ to the server.

Now, the view of the server in the ideal game is $\text{View}_{server}^{\text{Sim}} := (\text{pk}, r, \text{ct}'_f, \text{ct}'_{f^*})$. It is straightforward from the IND-CPA security of the FHE scheme that $(\text{ct}'_f, \text{ct}'_{f^*})$ is computationally indistinguishable from the ciphertext pair $(\text{ct}_f, \text{ct}_{f^*})$. Therefore, the views of the semi-honest server in the real game and the simulated game are indistinguishable. That is $\text{View}_{server}^{\Pi} \stackrel{c}{\approx} \text{View}_{server}^{\text{Sim}}$.

Below, we show the security against the semi-honest CSP.

Theorem VI.3: The proposed protocol Π is secure against the semi-honest server.

Proof: Note that the CSP receives the ciphertext ct_{s+r} of the masked similarity $s + r \in \mathbb{Z}_{q_p}$ from the server for some random $r \in \mathbb{Z}_{q_p}$. Therefore, the view of the CSP in the real game is $\text{View}_{CSP}^{\Pi} := (\text{pk}, \text{sk}, \text{ct}_{s+r})$, where r is a random integer over $\mathbb{Z}_{q_p}$. In the simulated game, the simulator Sim randomly samples a random integer $r' \in \mathbb{Z}_{q_p}$, and then sends $\text{ct}_{r'} := \text{FHE} \cdot \text{Enc}(r')$ to the CSP. Now, the view of the CSP in the ideal game is $\text{View}_{CSP}^{\text{Sim}} := (\text{pk}, \text{sk}, \text{ct}_{r'})$. since r is randomly distributed over $\mathbb{Z}_{q_p}$, the masked similarity $s + r \bmod q_p$ is randomly distributed over $\mathbb{Z}_{q_p}$. Therefore, the distribution of $s + r$ is the same as r' . Therefore, the views of the semi-honest CSP in the real game and the simulated game are indistinguishable. That is $\text{View}_{CSP}^{\Pi} \stackrel{c}{\approx} \text{View}_{CSP}^{\text{Sim}}$. ■

The above security analysis shows that the proposed protocol leaks no information to the semi-honest server and CSP. For the semi-honest client, the proposed protocol only leaks the verification result. This is what a face verification protocol is designed for. Remark that the random oracle in the above security proof is often replaced by the collision hash function in real-world applications.

VII. IMPLEMENTATIONS AND EVALUATIONS

From the description of the protocol presented in Section V.B, we know that the verification process consists of two main phases: similarity computation and threshold comparison. The

similarity computation phase mainly depends on the batch inner product technique given in Algorithm 1. The threshold comparison mostly depends on the hash-based comparison. Below, we first present the protocol parameter settings. Next, we present an experimental evaluation of the face verification protocol. Finally, we compare the main novel techniques introduced in this paper with the existing techniques.

A. Parameter Settings

Our experimental results³ are obtained on a laptop with an Intel Core i7-1165G7 CPU@2.8 GHz processor. To implement the proposed protocol, we use the well-known face datasets LFW [41], AgeDB [42] and CFP-FP [43], and use the state-of-the-art face recognition model ArcFace [21] to extract 512-dimensional feature vectors. Elements in the feature vectors are floating-point numbers. Fig. 6(a) shows the accuracy of the face verification protocol with different precisions. It is easy to observe that if we keep 2-decimal floating digits in the elements of feature vectors, then the recognition process keeps the same accuracy as the original, while the verification threshold $\hat{t}$ of ArcFace's cosine similarity equals 0.20. Note that the larger enlarging factor γ reduces the efficiency of FHE operations, e.g., encryptions, decryptions, and inner products. However, when the factor γ is too small, the accuracy of biometric verification will decrease. Thus, by keeping two decimal digits in the feature vector, we can keep the high accuracy and efficiency simultaneously. Thus, we set the magnification factor $\gamma = 100$ in the evaluations of our protocol. Fig. 6(c) and (d) show the time and space cost of homomorphic operation for different γ , which takes values in $10^5, 10^4, 10^3, 10^2$ and the plaintext modulus q_p takes values in $2^{58}, 2^{48}, 2^{36}, 2^{24}$. It is easy to see from Fig. 6(c) and (d) that when the $\gamma = 10^2$ and the plaintext modulus is 2^{24} , the time and space cost of the FHE operations are significantly better than other larger γ and plaintext modulus. Note that the plaintext modulus q_p should be larger than the enlarged similarity, so that the server correctly computes the encryption of the amplified similarity. Thus, using a larger γ will incur a larger plaintext modulus, and thus a larger scheme modulus. Therefore, smaller γ brings more efficient protocol.

Note that in the comparison process of the masked similarity-threshold, we use the shrinking factor p to control the size of the interval. If the value of p is too large, the accuracy of biometric verification will decrease. On the other hand, the smaller p keeps the accuracy of the verification process. In our experimental evaluations, we let the shrinking factor p take values in $\{1, 2, 4, 16, 64, 128, 1024, 2048\}$, which corresponds to the different number of hashes in the secure threshold comparison.

For homomorphic evaluation of the cosine similarity, the homomorphic operations run on the FHE library SEAL [39] with the BFV scheme. The underlying ring of the BFV scheme is set as follows: the ring dimension $n = 4096$, ring modulus q is of 109 b length, and the plaintext modulus is 2^{24} . In the threshold comparison phase, we use SHA256 to instantiate the cryptographic hash function. The performance evaluations of

³<https://github.com/jucing/BioFast>

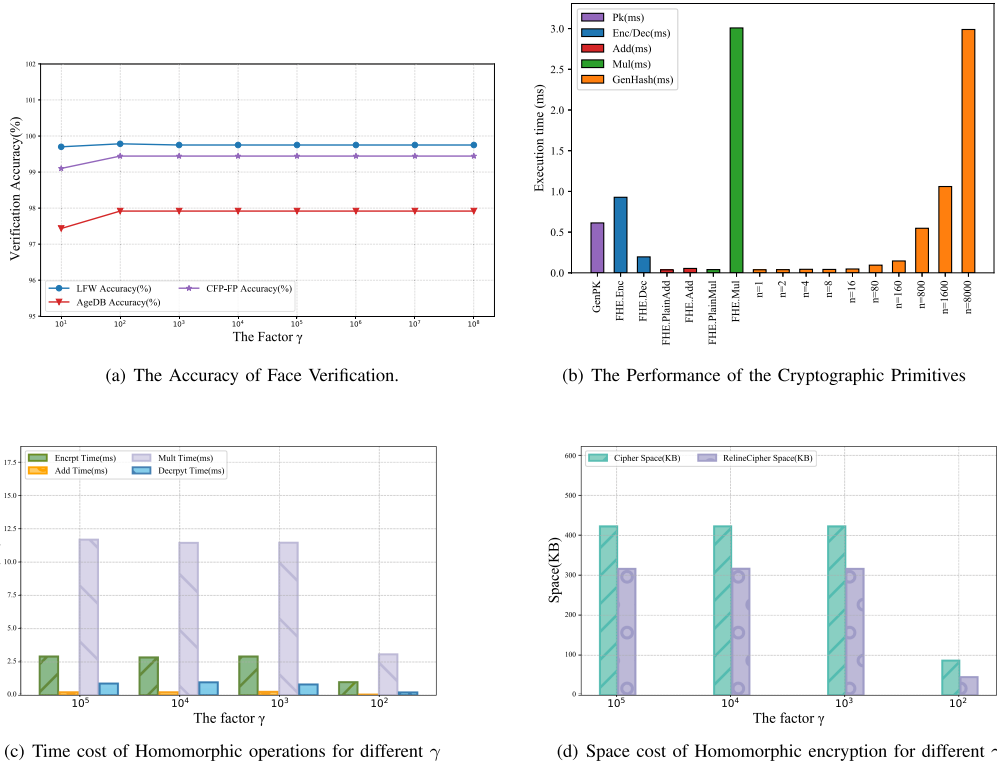


Fig. 6. The accuracy of face recognition and efficiency of cryptographic primitives.

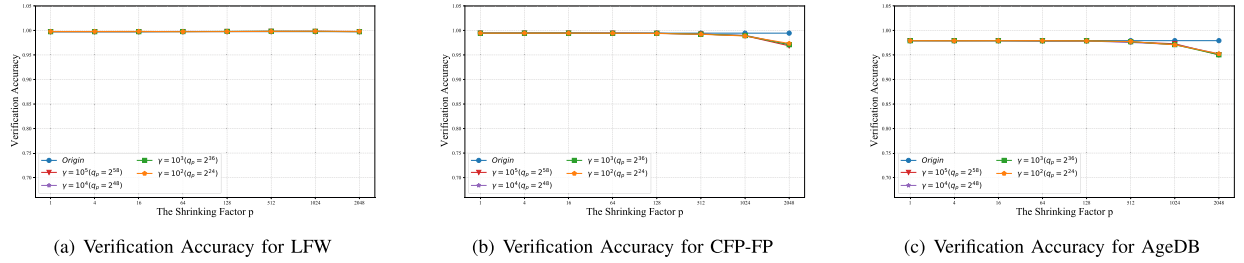


Fig. 7. Verification accuracy for different datasets, γ and shrinking factor p .

FHE operations are given in Fig. 6(b). The GenPK represents the public key generation process of the FHE scheme. The FHE.Enc (or FHE.Dec) represents the encryption (or decryption) algorithm of the FHE scheme. The FHE.PlainAdd (or FHE.PlainMul) represents the homomorphic addition (or multiplication) of an FHE ciphertext and a plain number. The FHE.Add (or FHE.Mul) represents the homomorphic addition (or multiplication) of two FHE ciphertexts.

The performance evaluations of the hash function with various amounts are given in Fig. 6(b), where the number of hashes are taken values in $\{1, 2, 4, 8, 16, 80, 160, 800, 1600, 8000\}$. Note that the threshold $\hat{t}$ is 0.20, and thus if the cosine similarity of two feature vectors is in $[0.20, 1]$, then we assert that two feature vectors are matched. From the above set of $\gamma = 100$ and shrinking factor p , we know that the server needs to compute $\left\lceil \frac{0.8 \cdot \gamma^2}{p} \right\rceil$ hashes.

Fig. 7 shows the verification accuracy for LFW [41], CFP-FP [43], and AgeDB [42] for the different γ values and shrinking factor p . In Fig. 7, "Origin" means plaintext verification without using any encryption techniques. Fig. 7 shows that the verification process almost maintains a high accuracy whenever γ equals 10^2 , 10^3 , 10^4 , or 10^5 for $p \leq 1000$. However, as we discussed above and showed in Fig. 6(c) and Fig. 6(d), smaller γ means smaller time and space cost. Besides, Fig. 7(a) shows that when p is greater than 1024, the verification accuracy starts to decline. In our protocol, we set $p = 128$ such that the protocol achieves high accuracy.

B. Experimental Evaluations of the Protocol

Below, we present the experimental evaluations of the enrollment phase and verification phase of our protocol in both time and space efficacy.

TABLE II
THE OPERATIONS AND THE COMMUNICATION OVERHEAD FOR ONE ENROLLMENT

Participants Process	FHE Operations			Communication Overhead
	Client	Server	CSP	
FHE Key request	-	-	1 · FHE.keyGen	1 · pk
Template storage	1 · FHE.Enc	1 · FHE.PlainMul + 1 · FHE.Add	-	1 · ct
Total	1 · FHE.Enc	1 · FHE.PlainMul + 1 · FHE.Add	1 · FHE.keyGen	1 · ct + 1 · pk

TABLE III
THE EVALUATION OF THE ENROLLMENT PHASE

Participants Process	Computation overhead (ms)			Communication overhead (KB)
	Client	Server	CSP	
FHE key request	-	-	0.613	130.95
Template storage	0.928	0.0936	-	86.4
Total	0.928	0.0936	0.613	217.35

TABLE IV
THE OPERATIONS AND THE COMMUNICATION OVERHEAD FOR ONE VERIFICATION

Participants Process	Computation overhead			Communication overhead
	Client	Server	CSP	
Similarity	1 · FHE.Enc	$\frac{1}{3} \cdot \text{FHE.PlainMul} + \frac{1}{2} \cdot \text{FHE.Mul}$	-	1 · ct
Threshold Comparison	-	$\left(\left\lceil \frac{\gamma^2 - t}{p} \right\rceil\right) \cdot \text{Hash}$	1 · FHE.Dec + 1 · Hash	$\frac{1}{2} \cdot ct + \left(\left\lceil \frac{\gamma^2 - t}{p} \right\rceil\right) \cdot \text{Hash}$
Total	1 · FHE.Enc	$\frac{1}{2} \cdot \text{FHE.PlainMul} + \frac{1}{2} \cdot \text{FHE.Mul} + \left(\left\lceil \frac{\gamma^2 - t}{p} \right\rceil\right) \cdot \text{Hash}$	1 · FHE.Dec + 1 · Hash	$\frac{3}{2} \cdot ct + \left(\left\lceil \frac{\gamma^2 - t}{p} \right\rceil\right) \cdot \text{Hash}$

a) *Evaluations of the Enrollment:* Note that, in the enrollment phase, the CSP runs the FHE key generation algorithm once for all and then sends the public key pk to the client as a response to the FHE public key query. Next, to store the feature templates in the server, the client sends an FHE ciphertext for each feature vector, and the server (as shown in the Algorithm 2) makes a plain monomial multiplication for per encrypted template feature, and accumulative addition (1 addition per encrypted feature vector). The FHE operations and the communication overheads are shown in Table II. The experimental evaluations of the enrollment phase is given in Table III.

b) *Evaluations of the Verification:* From the description of the verification process of the proposed protocol in the previous sections, the client first sends a ciphertext of the probe feature to the server, then the server homomorphically computes the similarity as in the Algorithm 3, which needs one plain multiplication of a monomial and a ciphertext homomorphic multiplication. Thereafter, the server sends the additive masked similarity to the CSP and sends the hashes of the integers in shrunk intervals to the client. After receiving the masked similarity, the CSP decrypts it and then sends the hash of the decrypted message to the client. After all, the client checks if the hash value from the CSP is in the set of hashes received from the server. The FHE operations and corresponding communication

TABLE V
THE EVALUATION OF VERIFICATION PHASE

Participants Process	Computation overhead (ms)			Communication overhead (KB)
	Client	Server	CSP	
Similarity	0.928	1.524	-	86.4
Threshold Comparison	-	0.0944	0.1973	25.189
Total	0.928	1.618	0.1973	111.589

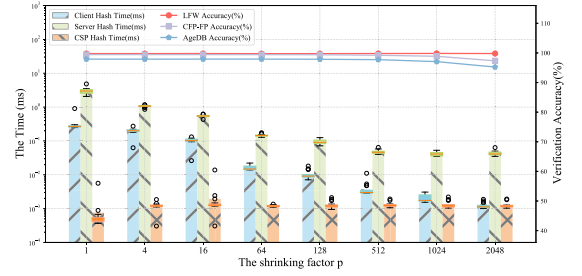


Fig. 8. The running time of our hash-based threshold-similarity comparisons.

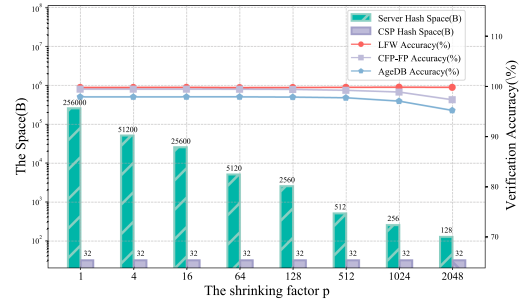


Fig. 9. The space cost of our hash-based threshold-similarity comparisons.

overheads are presented in Table IV. The concrete performance of the verification phase is given in Table V.

Remark VII.1: Note that the basic operations are much more efficient than the FHE homomorphic operations and hash evaluations, thus the computation overheads of those basic operations are ignored in Table IV.

The shrinking factor p introduces no additional error if we set $p = 1$. However, the resulted scheme needs to transmit large amount of hash values, and that are larger than the GC method. Therefore, we apply larger p in the rounding operation to decrease the communication overheads. As the Figs. 8 and 9 show the communication overheads and the time costs are decreases as the shrinking factor p increases, and thus the scheme becomes more efficient. It is not hard to see from Figs. 8 and 9 that when $p \leq 128$, the accuracy almost the as same as origin, and the accuracy declines for larger $p \geq 512$. Therefore, it is plausible to set $p = 128$ if considering the efficiency and the accuracy.

C. Comparisons of the Related Techniques

To compare our protocol with previous garbled circuit-based protocols, we use ABY library [44] with a bandwidth of

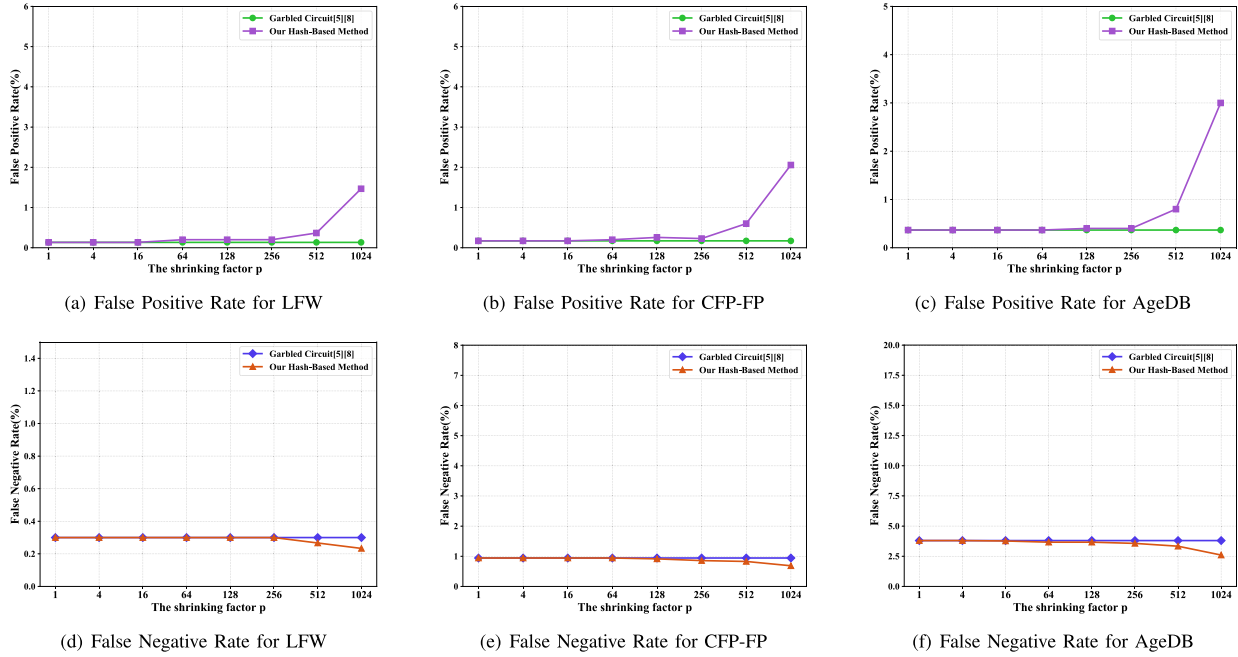


Fig. 10. False negative rate and false positive rate comparisons for various shrinking factor p and different datasets.

487.5 Mb/s local network. Intuitively, the garbled circuit-based protocol needs more frequent interactions because the garbled gate evaluation process needs to use oblivious transfer (OT). Thus, the quality of the communication network is much more inefficient than the local computation costs.

Fig. 10 show the false positive rate and false negative rate of garbled circuit-based approaches and our hash-based approach. Fig. 10(a)–(c) show that when the shrinking factor p increases, the false positive rate of our hash-based method increases. This is because the rounding operation has caused the acceptance of more rejected negative samples than the rejected positive samples. Note that the increment of the FPR is not only related to the data sets, but also with the similarity thresholds and shrinking factor p . This experimental result supports the theoretical results in Theorem IV.3. Fig. 10(d)–(f) show that when the shrinking factor p increases, the false negative rate of our method deviates significantly from the garbled circuit-based method. This is because the positive samples rejected by the model were corrected by the rounding operation. We also remark that the decrement of FNR comes at the cost of increment on the FPR. Note that these evaluation metrics considered both the inherit nature of the face models and the impact of the shrinking factor.

We compare these two methods both in communication overhead (Byte) and time (ms) in Table VI, where we use 32-bit length for garbled circuits, and GC(LAN) denotes the garbled circuit is used in WLAN environment, and GC(Local) denotes the case that the garbled circuit based similarity threshold comparison is evaluated locally. Table VI reconfirms the above intuition that the locally garbled evaluation of the threshold comparison circuit takes about 1 ms. In contrast, it takes dozens of milliseconds in the WLAN setting. This verifies the above intuition.

TABLE VI
COMPARISON BETWEEN THE GARBLED CIRCUIT-BASED APPROACH AND HASH-BASED APPROACH

	server (ms)	CSP (ms)	server (B)	CSP (B)	total (KB)
GC(LAN) [5], [8]	15.5469	11.938	2116	6647	8.56
GC(Local) [5], [8]	1.1471	1.1466	2114	6647	8.56
<i>Ours</i> (LAN)	0.0953	0.0027	2560	32	2.53
<i>Ours</i> (Local)	0.0944	0.0013	2560	32	2.53

Compared with the garbled circuit-based approaches, the communication frequency of our hash-based approach is much smaller. Since the communication frequency is almost optimal, our protocol is more robust regarding network quality. Table VI shows that when $p = 128$, our hash-based threshold-similarity comparison process only needs about 0.1 ms, which is almost $10\times$ better than that of garbled circuit-based approaches [5], [8]. In addition, the total communication overhead is about 2.5 KB, which is about $3\times$ better than that of garbled circuit-based approaches [5], [8].

We compare the similarity computation process of our work with the state-of-the-art lattice-based HE-based protocols in Table VII. Note that the identification protocols [4], [8], [12] use column strategy (refer to [8]) via encrypting each entry of the probe features, which significantly increases the communication and computation complexity of the similarity computation process. Therefore, we didn't list them in the above table. Also, there are many other approaches using different techniques, as our main focus in this work is post-quantum security, we didn't list them in the table. To be fair, we implement these lattice-based FHE based similarity computation Algorithms in our laptop with the same parameters as us.

TABLE VII
COMPARISON OF SIMILARITY COMPUTATION PROCESS

Performance Solutions	Theoretical Overhead Homomorphic Computation Costs	Experimental Overhead		Methods	
		Communication (KB)	Time (ms)	Metrics	Any Modulus
[15]	$k \cdot \text{FHE.Mul} + k \cdot \text{FHE.Sub} + (k-1) \cdot \text{FHE.Add}$	44265	1592	ED	✗
[7]	$1 \cdot \text{FHE.Mul} + 1 \cdot \text{FHE.Sub} + (k-1) \cdot \text{FHE.Add}$ $+ (k-1) \cdot \text{FHE.Rotation}$	86.48	274.54	ED	✗
[17] [‡]	$\frac{1}{\lfloor n/k \rfloor} \cdot \text{FHE.Mul} + \frac{1}{\lfloor n/k \rfloor} \cdot \text{FHE.Sub}$ $+ \frac{1}{\lfloor n/k \rfloor} \cdot (k-1) \cdot \text{FHE.Add}$ $+ \frac{1}{\lfloor n/k \rfloor} \cdot (k-1) \cdot \text{FHE.Rotation}$	10.80	33.63	ED	✗
[5], [13]	$1 \cdot \text{FHE.Mul} + 1 \cdot \text{FHE.Sub} + \log_2(k) \cdot \text{FHE.Add}$ $+ \log_2(k) \cdot \text{FHE.Rotation}$	86.48	8.44	ED	✗
[6], [14]	$1 \cdot \text{FHE.Mul} + \log_2(k) \cdot \text{FHE.Add}$ $+ \log_2(k) \cdot \text{FHE.Rotation}$	86.48	8.37	CosSim	✗
[45]	$1 \cdot \text{FHE.Mul}$	86.48	2.976	ED	✓
Ours	$\frac{1}{2} \cdot \text{FHE.PlainMul} + \frac{1}{2} \cdot \text{FHE.Mul}$	43.24	1.524	CosSim	✓

Since the homomorphic operations are the most inefficient part of the similarity computation process, we only list the homomorphic operation in this table. The column Any Modulus refers to the underlying similarity computation technique that can be applied to any modulus q of the ring $\mathbb{Z}_q[x]/(x^n + 1)$. In this table, the experimental results are obtained in the ring setting of $\mathbb{Z}_q[x]/(x^{4096} + 1)$.

‡: Note that the work of [17] used the PCA technique to compress the feature vectors to output compressed feature vectors of smaller lengths. Since all the above protocols support PCA compression, to be fair, we didn't use PCA compression and set $k = 512$ for all protocols in the above table, and the experimental overheads are averaged per feature. However, the number of homomorphic rotations in [17] is linear in the feature vector dimension, and thus, the running time is relatively inefficient.

Note that the work of [15] doesn't consider packing techniques, and thus, the communication and computation costs are very large. Although the work of [7] have used the packing techniques of lattice-based HE schemes, the underlying method needs $(k-1)$ homomorphic rotation, where k is the dimension of the feature vectors. The work of [17] used the PCA technique to output compressed features vectors of length $\bar{k}$ and use packing techniques to pack more vectors into one ring element, and thus communication cost is averaged to each feature. However, the number of homomorphic rotations in their method is linear in $\bar{k}$, and thus the running time is relatively inefficient. The works of [5], [6], [13], [14] need ω homomorphic rotations. Note that the work of [45] is requires only 1 FHE.Sub (subtraction) and 1 FHE.Mul (multiplication) for computing one similarity. However, their work not supports packing like ours. In other words, our packed similarity computation technique supports embedding more feature vectors into one ciphertext and thus can save more space. Furthermore, our similarity computation process only requires a homomorphic multiplication to compute multiple inner products. Thus, as the Section IV.B shows, when the ring dimension $n = 4096$, we can embed two 512-dimensional feature vector into one ciphertext. Thus, as Table VII shows that our similarity computation process is almost $2\times$ efficient than the work of [45]. Table VII shows that our similarity computation technique doesn't require homomorphic rotations, and thus, the running time is very efficient with the comparable communication cost. In addition, our technique can perform on any ring modulus of the cyclotomic ring $\mathbb{Z}_q[x]/(x^n + 1)$. This cyclotomic ring is very important since the lattice-based homomorphic encryption

schemes are designed on this kind of ring. In sum, Our similarity computation process achieves significantly better time efficiency than previous post-quantum HE-based approaches with comparable communication overhead. These experimental results strongly support our theoretical analysis in the previous sections.

VIII. CONCLUSION

In this paper, we propose a more efficient privacy-preserving face recognition protocol based on homomorphic encryption and cryptographic hash function. In the proposed protocol, the server and CSP cannot obtain any user's face data and verification result while the client only knows the verification result. To achieve this efficient protocol, we develop a new coefficient packing technique to compute similarity and design a new hash based threshold-similarity comparison method. The experimental results strongly supports our theoretical analysis and intuitions. The developed techniques can be applied to other biometric verification protocols straightforwardly, and improves the efficiency of the state of the art biometric verification protocols.

REFERENCES

- [1] P. Gasti, J. Sedenka, Q. Yang, G. Zhou, and K. S. Balagani, "Secure, fast, and energy-efficient outsourced authentication for smartphones," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 11, pp. 2556–2571, Nov. 2016.
- [2] H. Gunasinghe and E. Bertino, "PrivBioMTauth: Privacy preserving biometrics-based and user centric protocol for user authentication from mobile phones," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 4, pp. 1042–1057, Apr. 2018.

- [3] J.-H. Im, S.-Y. Jeon, and M.-K. Lee, "Practical privacy-preserving face authentication for smartphones secure against malicious clients," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 2386–2401, 2020.
- [4] J. J. Engelsma, A. K. Jain, and V. N. Boddeti, "HERS: Homomorphically encrypted representation search," *IEEE Trans. Biom., Behav., Ident. Sci.*, vol. 4, no. 3, pp. 349–360, Jul. 2020.
- [5] H. Huang and L. Wang, "Efficient privacy-preserving face verification scheme," *J. Inf. Secur. Appl.*, vol. 63, 2021, Art. no. 103055.
- [6] V. N. Boddeti, "Secure face matching using fully homomorphic encryption," in *Proc. IEEE 9th Int. Conf. Biometrics Theory Appl. Syst.*, 2018, pp. 1–10.
- [7] P. Drozdowski, N. Buchmann, C. Rathgeb, M. Margraf, and C. Busch, "On the application of homomorphic encryption to face identification," *Proc. Int. Conf. Biometrics Special Int. Group*, 2019, pp. 1–5.
- [8] H. Huang and L. Wang, "Efficient privacy-preserving face identification protocol," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2632–2641, Jul./Aug. 2023.
- [9] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) fully homomorphic encryption without bootstrapping," in *Proc. Innov. Theor. Comput. Sci.*, S. Goldwasser, Ed. Cambridge, MA, USA, ACM, 2012, pp. 309–325, doi: [10.1145/2090236.2090262](https://doi.org/10.1145/2090236.2090262).
- [10] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, 2012, Art. no. 144. [Online]. Available: <http://eprint.iacr.org/2012/144>
- [11] I. Damgård, M. Geisler, and M. Krøigaard, "A correction to 'efficient and secure comparison for on-line auctions,'" *Int. J. Appl. Cryptography*, vol. 1, no. 4, pp. 323–324, 2009, doi: [10.1504/IJACT.2009.028031](https://doi.org/10.1504/IJACT.2009.028031).
- [12] S. Hu, M. Li, Q. Wang, S. S. M. Chow, and M. Du, "Outsourced biometric identification with privacy," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 10, pp. 2448–2463, Oct. 2018, doi: [10.1109/TIFS.2018.2819128](https://doi.org/10.1109/TIFS.2018.2819128).
- [13] D. Osorio-Roig, C. Rathgeb, P. Drozdowski, and C. Busch, "Stable hash generation for efficient privacy-preserving face identification," *IEEE Trans. Biom., Behav., Ident. Sci.*, vol. 4, no. 3, pp. 333–348, Jul. 2022.
- [14] S. Jumonji, K. Sakai, M.-T. Sun, and W.-S. Ku, "Privacy-preserving collaborative filtering using fully homomorphic encryption," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2961–2974, Mar. 2023.
- [15] J. R. Troncoso-Pastoriza, D. González-Jiménez, and F. Pérez-González, "Fully private noninteractive face verification," *IEEE Trans. Inf. Forensics Secur.*, vol. 8, no. 7, pp. 1101–1114, Jul. 2013.
- [16] J. Sedenka, S. Govindarajan, P. Gasti, and K. S. Balagani, "Secure outsourced biometric authentication with performance evaluation on smartphones," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 2, pp. 384–396, Feb. 2015.
- [17] P. Bauspies, J. Ö. Ólafsson, J. Kolberg, P. Drozdowski, C. Rathgeb, and C. Busch, "Improved homomorphically encrypted biometric identification using coefficient packing," in *Proc. Int. Workshop Biometrics Forensics*, 2022, pp. 1–6.
- [18] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 815–823.
- [19] Y. Wen, K. Zhang, Z. Li, and Y. Qiao, "A discriminative feature learning approach for deep face recognition," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 499–515.
- [20] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep hypersphere embedding for face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6738–6746.
- [21] J. Deng, J. Guo, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4685–4694.
- [22] H. Wang et al., "Cosface: Large margin cosine loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5265–5274.
- [23] Z. Erkin, M. Franz, J. Guajardo, S. Katzenbeisser, R. L. Lagendijk, and T. Toft, "Privacy-preserving face recognition," in *Proc. Int. Symp. Privacy Enhancing Technol.*, 2009, pp. 235–253.
- [24] G. Pradel and C. J. Mitchell, "Privacy-preserving biometric matching using homomorphic encryption," in *Proc. IEEE 20th Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2021, pp. 494–505.
- [25] K. Zhou and J. Ren, "PassBio: Privacy-preserving user-centric biometric authentication," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 12, pp. 3050–3063, Dec. 2017.
- [26] A.-R. Sadeghi, T. Schneider, and I. Wehrenberg, "Efficient privacy-preserving face recognition," *IACR Cryptol. ePrint Arch.*, vol. 2009, 2009, Art. no. 507.
- [27] R. L. Lagendijk, Z. Erkin, and M. Barni, "Encrypted signal processing for privacy protection: Conveying the utility of homomorphic encryption and multiparty computation," *IEEE Signal Process. Mag.*, vol. 30, no. 1, pp. 82–105, Jan. 2013.
- [28] A. Bassit, F. Hahn, J. Peeters, T. Kevenaar, R. Veldhuis, and A. Peter, "Fast and accurate likelihood ratio-based biometric verification secure against malicious adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 5045–5060, 2021.
- [29] A. Abidin, "On privacy-preserving biometric authentication," in *Proc. Conf. Inf. Secur. Cryptol.*, 2016, pp. 169–186.
- [30] J. Kolberg, P. Drozdowski, M. Gomez-Barrero, C. Rathgeb, and C. Busch, "Efficiency analysis of post-quantum-secure face template protection schemes based on homomorphic encryption," in *Proc. Int. Conf. Biometrics Special Int. Group*, 2020, pp. 1–4.
- [31] Y. Ma, L. Wu, X. Gu, J. He, and Z. Yang, "A secure face-verification scheme based on homomorphic encryption and deep neural networks," *IEEE Access*, vol. 5, pp. 16532–16538, 2017.
- [32] J.-B. Ji et al., "Privacy-preserving face recognition with learnable privacy budgets in frequency domain," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 475–491.
- [33] G. Zhao, Q. Jiang, D. Wang, X. Ma, and X. Li, "Deep hashing based cancelable multi-biometric template protection," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3751–3767, Jul./Aug. 2023.
- [34] J. Lei, Q. Pei, Y. Wang, W. Sun, and X. Liu, "PrivFace: Fast privacy-preserving face authentication with revocable and reusable biometric credentials," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 5, pp. 3101–3112, Sep./Oct. 2022, doi: [10.1109/TDSC.2021.3083970](https://doi.org/10.1109/TDSC.2021.3083970).
- [35] Y. Ren, Z. Song, S. Sun, J. K. Liu, and G. Feng, "Outsourcing LDA-based face recognition to an untrusted cloud," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 3, pp. 2058–2070, May/Jun. 2023, doi: [10.1109/TDSC.2022.3172143](https://doi.org/10.1109/TDSC.2022.3172143).
- [36] Y. Zhao and S. S. M. Chow, "Can you find the one for me? privacy-preserving matchmaking via threshold PSI," in *Proc. Workshop Privacy Elect. Soc.*, Toronto, ON, Canada, Oct. 2018, pp. 54–65, doi: [10.1145/3267323.3268965](https://doi.org/10.1145/3267323.3268965).
- [37] L. K. L. Ng and S. S. M. Chow, "Gforce: GPU-friendly oblivious and rapid neural network inference," in *Proc. 30th USENIX Secur. Symp.*, M. D. Bailey and R. Greenstadt, Eds., 2021, pp. 2147–2164. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/ng>
- [38] Á. Kiss, M. Naderpour, J. Liu, N. Asokan, and T. Schneider, "SOK: Modular and efficient private decision tree evaluation," in *Proc. Privacy Enhancing Technol.*, 2019, vol. 2019, no. 2, pp. 187–208, doi: [10.2478/popets-2019-0026](https://doi.org/10.2478/popets-2019-0026).
- [39] "SEAL Microsoft (release 3.0)," Oct. 2018, Microsoft Research, Redmond, WA, USA. [Online]. Available: <http://sealcrypto.org>
- [40] N. NIST and M. Dworkin, "SHA-3 Standard: Permutation-based hash and extendable-output functions," Federal Inf. Process. Stds. (NIST FIPS), National Institute of Standards and Technology, Gaithersburg, MD, 2015, doi: [10.6028/NIST.FIPS.202](https://doi.org/10.6028/NIST.FIPS.202). [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=919061
- [41] G. B. Huang, M. A. Mattar, T. L. Berg, and E. Learned-Miller, "Labeled faces in the wild: A database for studying face recognition in unconstrained environments," in *Proc. Workshop Faces 'Real-Life' Images: Detection, Alignment, Recognit.*, 2008.
- [42] S. Moschoglou, A. Papaioannou, C. Sagonas, J. Deng, I. Kotsia, and S. Zafeiriou, "AgeDB: The first manually collected, in-the-wild age database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2017, pp. 1997–2005.
- [43] S. Sengupta, J.-C. Chen, C. Castillo, V. M. Patel, R. Chellappa, and D. W. Jacobs, "Frontal to profile face verification in the wild," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2016, pp. 1–9.
- [44] D. Demmler, T. Schneider, and M. Zohner, "ABY-A framework for efficient mixed-protocol secure two-party computation," in *Proc. NDSS*, 2015, pp. 1–15.
- [45] H. Tamiya, T. Isshiki, K. Mori, S. Obana, and T. Ohki, "Improved post-quantum-secure face template protection system based on packed homomorphic encryption," in *Proc. Int. Conf. Biometrics Special Int. Group*, 2021, pp. 1–5.



Parhat Abila received the PhD degree in cyber security from the Institute of Information Engineering, Chinese Academy of Sciences and University of Chinese Academy of Sciences, China, in 2022. His research papers are published in international conferences, such as TCC, CT-RSA, SAC, and journals such as *IEEE Transactions on Information Forensics and Security*, *Transactions on Computers*. His research interests include data security, cryptography, and lattice-based cryptography.



Yongqiang Li received the BS and MS degrees in mathematics from Beijing Normal University, Beijing, China, in 2005 and 2008, respectively, and the PhD degree in information security from the Institute of Software, Chinese Academy of Sciences, China, in 2012. He is currently an associate professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research papers are published in top-ranked conferences and journals such as CHES, Asiaticrypt, *IEEE Transactions on Information Theory*. His research interests include symmetric cryptography and related areas.



Rongbin Huang received the BS degree from the School of Software, South China Normal University, Guangzhou, China, in 2023, where he is currently working toward the MS degree with the School of Software. His research interests include biometric recognition and fully homomorphic encryption.



Mingsheng Wang received the PhD degree from Beijing Normal University, Beijing, China, in 1994. He is currently a professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing. His research papers are published in top-ranked conferences and journals such as EUROCRYPT, CHES, Asiaticrypt, *IEEE Transactions on Information Theory*. His research interests include computational algebra, cryptography, and information security.